



CONTROL HORARIO

Sistema de gestión de fichajes

LIBRO BACKEND

Documentación técnica del backend
del sistema de control horario



GESTIÓN DE USUARIOS

Roles, permisos y administración



CONTROL DE FICHAJES

Registro de entradas y salidas



GEOLOCALIZACIÓN

Control de ubicación en tiempo real



SEGURIDAD

Autenticación, JWT y roles



AUDITORÍA E INTEGRIDAD

Trazabilidad y hash de datos



EXPORTACIONES

PDF y CSV



```
1
2 import org.springframework.web.bind.annotation.*;
3 import org.springframework.http.ResponseEntity;
4 import java.util.*;
5
6 @RestController
7 @RequestMapping("/api/auth")
8 public class AuthController {
9
10     @PostMapping("/login")
11     public ResponseEntity<> login(@RequestBody LoginRequest request) {
12         if (authentication) {
13             String token = jwtService.generateToken(user);
14             return ResponseEntity.ok(new LoginResponse(token));
15         }
16         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
17     }
18 }
```


Libro Backend – Control Horario

ÍNDICE:

1. Introducción	pág.- 2
2. Tecnologías utilizadas	pág.- 2
3. Arquitectura del sistema	pág.- 3
4. Base de datos	pág.- 7
5. Autenticación y seguridad	pág.- 10
6. Gestión de usuarios	pág.- 13
7. Fichajes (control horario)	pág.- 17
8. Geolocalización	pág.- 19
9. Auditoría del sistema	pág.- 21
10. Integridad de datos (hash)	pág.- 23
11. Exportaciones (PDF/CSV)	pág.- 25
12. Flujo general de la aplicación	pág.- 27
13. Decisiones técnicas	pág.- 29
14. Posibles mejoras	pág.- 32

1. Introducción

Este proyecto es una aplicación de **control horario**, pensada para que los usuarios puedan registrar sus horas de trabajo de forma sencilla, y para que un administrador tenga control y visibilidad sobre esos registros.

La aplicación permite:

- Registrar entradas y salidas (fichajes)
- Consultar el tiempo trabajado
- Gestionar usuarios (activar, desactivar, roles...)
- Controlar fichajes abiertos o modificados
- Exportar información en diferentes formatos

A nivel técnico, el backend está construido con una arquitectura moderna basada en APIs, donde toda la lógica se ejecuta en el servidor y el frontend se comunica mediante peticiones HTTP.

El objetivo principal no ha sido solo que funcione, sino que sea:

- Seguro (autenticación, validaciones, control de accesos)
- Escalable (estructura clara y fácil de ampliar)
- Mantenible (código entendible para cualquier compañero)

Además, se han añadido funcionalidades importantes como:

- Control de acceso por roles (usuario/admin)
- Auditoría de acciones críticas
- Geolocalización en fichajes
- Sistema de recuperación de contraseña
- Protección frente a manipulaciones (hash en fichajes)

En resumen, no es solo una app de fichar, sino una base sólida que podría evolucionar fácilmente a un producto real.

2. Tecnologías utilizadas

El backend de la aplicación está construido usando tecnologías modernas del ecosistema web, buscando un equilibrio entre facilidad de desarrollo, rendimiento y escalabilidad.

Backend

El backend está desarrollado con Next.js, utilizando su sistema de rutas para crear una API.

Aunque Next.js se suele usar para frontend, en este caso se aprovecha como servidor para gestionar toda la lógica de la aplicación mediante endpoints (rutas API).

Se utiliza TypeScript, lo que permite trabajar con tipos y evitar muchos errores comunes, haciendo el código más robusto y fácil de mantener.

Base de datos

La base de datos utilizada es PostgreSQL, una base de datos relacional muy potente y ampliamente usada en entornos profesionales.

Para interactuar con la base de datos se utiliza Prisma, que actúa como intermediario entre el código y la base de datos.

Esto permite:

- Trabajar con objetos en lugar de SQL directo
- Tener el modelo de datos bien definido
- Facilitar cambios y mantenimiento

Además, se utiliza Prisma Studio para visualizar y gestionar los datos de forma gráfica durante el desarrollo.

Autenticación y seguridad

Para la autenticación se utilizan tokens JWT mediante la librería jsonwebtoken.

Esto permite que, una vez que el usuario inicia sesión, pueda hacer peticiones autenticadas sin tener que enviar su contraseña en cada solicitud.

Las contraseñas se almacenan de forma segura utilizando bcrypt, que aplica un sistema de hash para evitar guardar contraseñas en texto plano.

Envío de correos

Para funcionalidades como la recuperación de contraseña se utiliza Nodemailer junto con Postmark.

Esto permite enviar correos reales desde la aplicación, por ejemplo:

- Enlaces para restablecer contraseña
- Notificaciones básicas

Entorno de desarrollo

Durante el desarrollo se utiliza Docker para ejecutar la base de datos en local, facilitando la configuración y evitando problemas de entorno.

Resumen rápido

En pocas palabras:

- Next.js → servidor/backend
- TypeScript → seguridad en el código
- PostgreSQL → base de datos
- Prisma → conexión con la BD
- JWT + bcrypt → seguridad
- Nodemailer + Postmark → correos
- Docker → entorno local

3. Arquitectura del sistema

La aplicación sigue una arquitectura basada en **API**, donde el backend se encarga de toda la lógica y el frontend solo consume datos mediante peticiones.

En otras palabras:

- El frontend pide
- El backend decide
- La base de datos guarda

Estructura general del proyecto

El proyecto está dividido en dos partes:

- **Frontend (Angular)** → interfaz de usuario
- **Backend (Next.js)** → lógica y acceso a datos

Ambos están separados en carpetas distintas (frontend y backend) y se comunican mediante peticiones HTTP.

Esto permite trabajar cada parte por separado sin romper la otra.

Estructura real del backend

Dentro del backend, el proyecto sigue una estructura basada en rutas de Next.js.

Ejemplo simplificado:

```
/src/app/api/  
├── auth/  
│   ├── login/route.ts  
│   ├── me/route.ts  
│   └── reset-password/route.ts  
├── fichaje/  
│   ├── start/route.ts  
│   ├── stop/route.ts  
│   └── status/route.ts  
└── admin/  
    ├── users/route.ts  
    └── time-entries/route.ts
```

- Cada carpeta representa una funcionalidad
- Cada route.ts es un endpoint

Esto hace que sea muy fácil ubicar dónde está cada cosa.

Cómo funciona una ruta (endpoint)

Cada endpoint sigue más o menos el mismo flujo:

1. Recibe la petición (GET, POST, etc.)
2. Valida datos (body, params...)
3. Verifica permisos (usuario o admin)
4. Ejecuta lógica
5. Accede a la base de datos con Prisma
6. Devuelve respuesta

Esto hace que todas las rutas tengan un comportamiento consistente.

Control de acceso

Para evitar repetir código en cada endpoint, se usan funciones comunes:

- `requireAuth` → valida que el usuario tenga token válido
- `requireAdmin` → además comprueba si es administrador

Estas funciones se encargan de:

- Leer el token JWT
- Verificarlo
- Obtener el usuario

- Así todas las rutas están protegidas de forma uniforme.

Separación de responsabilidades

Aunque no hay capas formales tipo DAO/Service, sí hay una separación clara:

- La **ruta** gestiona la petición
- La **lógica** decide qué hacer
- Prisma se encarga de la base de datos

Esto evita mezclar todo en un mismo sitio y facilita el mantenimiento.

Ejemplo: iniciar fichaje

1. El usuario pulsa "Fichar"
2. El frontend llama a `/api/fichaje/start`
3. El backend:
 - verifica el token
 - comprueba que no haya fichaje abierto
 - guarda el fichaje
4. Devuelve el resultado

Decisiones tomadas

Se ha optado por este enfoque porque:

- Es fácil de entender para cualquier desarrollador
- Permite añadir nuevas rutas sin complicaciones
- Escala bien si la app crece
- Sigue una estructura moderna muy utilizada hoy en día

Organización interna de `src/lib`

Durante la revisión final del backend se reorganizaron las utilidades internas dentro de `src/lib`.

La estructura quedó separada por responsabilidades:

src/lib/

auth/

auth.ts

authRequest.ts

jwt.ts

rate-limit.ts

logging/

audit.ts

mail/

mail.ts

password-reset.ts

security/

password.ts

time-entry-integrity.ts

selects/

user.select.ts

http.ts

prisma.ts

url.ts

Esta organización ayuda a que el código sea más fácil de mantener.

Por ejemplo:

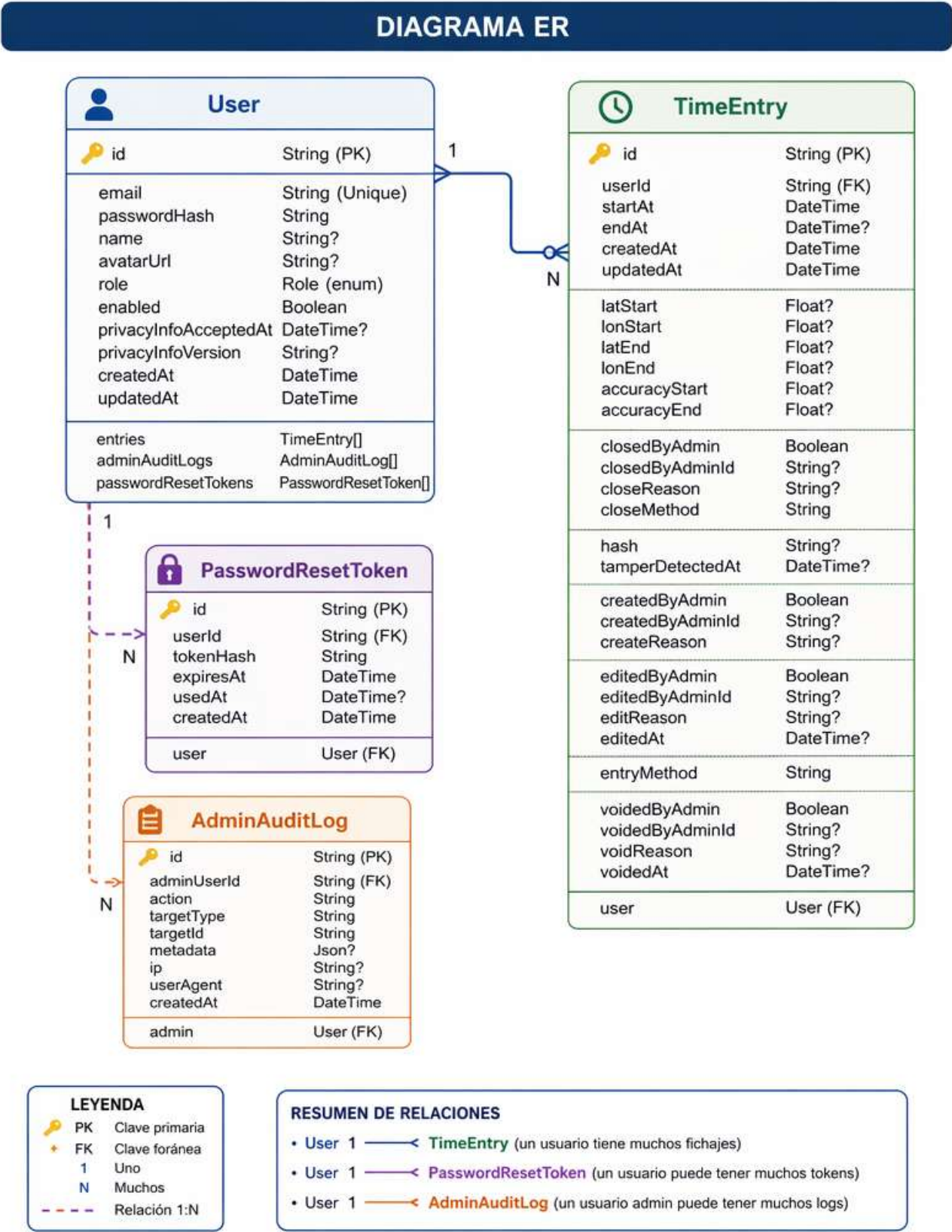
- Todo lo relacionado con autenticación está en auth
- La auditoría está en logging
- Los correos están en mail
- La seguridad de contraseñas e integridad de fichajes está en security
- Los selectores reutilizables de usuario están en selects

Esto evita tener utilidades mezcladas y facilita que un compañero nuevo encuentre rápido dónde está cada cosa.

4. Base de datos

Diagrama de la base de datos

Aquí tienes una representación visual de cómo están organizadas las entidades y sus relaciones:



Cómo entender el diagrama

La base de datos gira principalmente en torno al modelo **User**, que es el centro de todo. Desde ahí salen varias relaciones importantes:

Usuario → Fichajes

Un usuario puede tener muchos fichajes.

.-Ejemplo:

- Un empleado ficha cada día → muchos registros

Usuario → Tokens de recuperación

Un usuario puede tener varios tokens de recuperación de contraseña.

- Esto permite:

- Solicitar recuperación varias veces
- Controlar expiración y uso

Usuario → Auditoría

Un usuario administrador puede generar múltiples registros de auditoría.

.-Esto se usa para:

- Registrar acciones importantes
- Tener trazabilidad (quién hizo qué)

Fichajes

El modelo TimeEntry es el más completo y donde está gran parte de la lógica:

Incluye:

- Horas de inicio y fin
- Geolocalización
- Control de acciones de admin
- Detección de manipulación (hash)
- Historial de ediciones

Esto convierte un simple fichaje en un registro muy completo y seguro.

Ejemplo real del schema.prisma

A continuación se muestra una parte real del esquema usado en el proyecto:

EJEMPLO REAL DE SCHEMA.PRISMA

```
// ENUM
enum Role {
  ADMIN
  USER
}

// MODELOS
model User {
  id                String      @id @default(uuid())
  email             String      @unique
  passwordHash      String
  name              String?
  avatarUrl         String?
  role              Role        @default(USER)
  enabled           Boolean     @default(false)

  privacyInfoAcceptedAt DateTime?
  privacyInfoVersion String?

  createdAt         DateTime     @default(now())
  updatedAt         DateTime     @updatedAt

  entries           TimeEntry[]
  adminAuditLogs    AdminAuditLog[]
  passwordResetTokens PasswordResetToken[]
}

model PasswordResetToken {
  id                String      @id @default(uuid())
  userId            String
  tokenHash         String
  expiresAt         DateTime
  usedAt            DateTime?
  createdAt         DateTime     @default(now())

  user             User         @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([expiresAt])
}

model TimeEntry {
  id                String      @id @default(uuid())
  userId            String
  startAt           DateTime
  endAt             DateTime?
  createdAt         DateTime     @default(now())
  updatedAt         DateTime     @updatedAt
  latStart          Float?
  lonStart          Float?
  latEnd            Float?
  lonEnd            Float?
  accuracyStart     Float?
  accuracyEnd       Float?
  closedByAdmin     Boolean     @default(false)
  closedByAdminId   String?
  closeReason       String?
  closeMethod       String     @default("NORMAL")
  hash              String?
  tamperDetectedAt  DateTime?
  createdBByAdmin   Boolean     @default(false)
  createdByAdminId  String?
  createReason      String?
  editedByAdmin     Boolean     @default(false)
  editedByAdminId   String?
  editReason        String?
  editedAt          DateTime?
  entryMethod       String     @default("USER")
  voidedByAdmin     Boolean     @default(false)
  voidedByAdminId   String?
  voidReason        String?
  voidedAt          DateTime?
  user             User         @relation(fields: [userId], references: [id])
  @@index([userId, startAt])
}

model AdminAuditLog {
  id                String      @id @default(uuid())
  adminUserId       String
  action            String
  targetType         String
  targetId          String
  metadata          Json?
  ip                String?
  userAgent          String?
  createdAt         DateTime     @default(now())
}

admin User @relation(fields: [adminUserId], references: [id])
```

5. Autenticación y seguridad

La aplicación cuenta con un sistema de autenticación basado en tokens, junto con varias medidas de seguridad para proteger tanto a los usuarios como a los datos.

La idea es simple:

- Solo entra quien debe
- Cada uno ve lo que le corresponde
- Y todo lo sensible está protegido

Inicio de sesión (login)

La autenticación se basa en tokens JWT usando jsonwebtoken.

Cuando un usuario inicia sesión:

1. Introduce email y contraseña
2. El backend busca el usuario
3. Compara la contraseña con el hash almacenado usando bcrypt
4. Si todo es correcto → genera un token JWT
5. Ese token se devuelve al frontend

A partir de ahí, el usuario ya está autenticado.

Uso del token (cómo se mantiene la sesión)

El token JWT se envía en cada petición dentro de la cabecera:

Authorization: Bearer <token>

El backend lo valida en cada endpoint protegido.

- Si el token no es válido → acceso denegado
- Si es válido → se identifica al usuario

Protección de contraseñas

Las contraseñas **nunca se guardan en texto plano**.

Se utiliza bcrypt para:

- Encriptar la contraseña al registrarse
- Compararla de forma segura en el login

- Esto evita que, aunque alguien acceda a la base de datos, pueda ver contraseñas reales.

Control de acceso (roles)

El sistema distingue entre dos tipos de usuario:

- USER → usuario normal
- ADMIN → administrador

Dependiendo del rol:

- Un usuario solo puede ver y gestionar sus propios datos
- Un admin puede:
- Gestionar usuarios
- Ver todos los fichajes
- Modificar registros

Esto se controla con funciones como:

- `requireAuth` → usuario autenticado
- `requireAdmin` → usuario administrador

Revisión final de rutas protegidas

En la revisión final del backend se comprobó que las rutas sensibles estuvieran protegidas correctamente. Las rutas de administración usan `requireAdmin`, por ejemplo:

`/api/admin/*`

Las rutas privadas de usuario usan `requireAuth`, como:

`/api/auth/change-password`
`/api/auth/me`
`/api/auth/me/avatar`
`/api/auth/privacy-ack`
`/api/fichaje/*`

Las rutas públicas se mantienen accesibles sin token porque son necesarias antes de iniciar sesión:

`/api/auth/login`
`/api/auth/register`
`/api/auth/forgot-password`
`/api/auth/reset-password`

De esta forma, el backend controla realmente quién puede acceder a cada funcionalidad, independientemente de lo que muestre el frontend.

Protección frente a ataques básicos

Se han añadido medidas para evitar abusos:

Rate limiting (login)

El endpoint de login tiene un límite de intentos:

Ejemplo:

- Máximo 5 intentos por minuto

Esto evita ataques de fuerza bruta.

Respuestas neutras

En acciones como “recuperar contraseña”:

- El sistema **no revela si el email existe o no**

Siempre responde algo como:

"Si el email está registrado, recibirás un correo"

- Esto evita que alguien pueda descubrir usuarios válidos.

Respuestas de error más seguras

También se revisaron las respuestas de error para evitar mostrar detalles internos al cliente.

Antes, algunas respuestas podían devolver información técnica:

```
{  
  "message": "Internal Server Error",  
  "detail": "texto interno"  
}
```

Ahora se devuelve una respuesta más segura:

```
{  
  "message": "Error interno del servidor"  
}
```

Los detalles técnicos se mantienen en los logs internos, pero no se exponen al usuario ni al frontend.

Esto reduce el riesgo de mostrar información sensible o pistas sobre el funcionamiento interno del backend.

Recuperación de contraseña

El sistema incluye un flujo completo de recuperación:

1. El usuario solicita recuperación
2. Se genera un token único
3. Se guarda en la base de datos (en hash)
4. Se envía por correo con Nodemailer y Postmark
5. El usuario accede al enlace
6. Puede cambiar su contraseña

El token:

- Caduca (1 hora)
- Solo se puede usar una vez

Validaciones de seguridad

Las contraseñas deben cumplir:

- Mínimo 8 caracteres
- Al menos una mayúscula
- Al menos una minúscula
- Al menos un número

Esto se valida tanto en frontend como en backend.

6. Gestión de usuarios

La gestión de usuarios es una de las partes clave del backend, ya que controla quién puede acceder a la aplicación y qué permisos tiene cada persona dentro del sistema.

No se trata solo de crear usuarios, sino también de poder administrarlos de forma segura y ordenada.

Qué se guarda de cada usuario

Cada usuario tiene una serie de datos básicos y otros campos más orientados al control interno de la aplicación.

Los más importantes son:

- id → identificador único
- email → correo del usuario
- passwordHash → contraseña protegida
- name → nombre visible
- avatarUrl → imagen de perfil
- role → tipo de usuario (ADMIN o USER)
- enabled → indica si la cuenta está activa
- privacyInfoAcceptedAt → fecha de aceptación de la información de privacidad
- privacyInfoVersion → versión del texto legal aceptado
- createdAt y updatedAt → control de creación y actualización

Esto permite tener no solo usuarios funcionales, sino también trazabilidad y cierta base para temas legales o de cumplimiento.

Selectores reutilizables de usuario

Durante la revisión final se detectó que varios endpoints devolvían el objeto user con estructuras diferentes. Esto podía provocar inconsistencias en el frontend, por ejemplo al trabajar con privacidad, avatar o datos del usuario autenticado.

Para solucionarlo se creó un selector reutilizable llamado:
userPublicSelect

Ubicación:
src/lib/selects/user.select.ts

Este selector define qué campos públicos del usuario puede devolver el backend.
Campos incluidos:

id
email
name
avatarUrl
role
enabled
privacyInfoAcceptedAt

privacyInfoVersion
createdAt
updatedAt

Se usa en rutas como:

auth/login
auth/me
auth/me/avatar
auth/privacy-ack
auth/register

Gracias a esto, el frontend recibe siempre el mismo formato de usuario y se evitan errores por diferencias entre endpoints.

Selector para administración

Además del selector público, se añadió:
userAdminSelect

Actualmente se basa en userPublicSelect, pero está preparado para añadir en el futuro campos exclusivos para administración.

Por ejemplo:

- Último login
- Incidencias
- Departamento
- Métricas internas

Se usa en rutas como:

admin/users
admin/users/[id]

Esto permite separar claramente qué información puede recibir un usuario normal y qué información puede recibir un administrador.

Alta de usuarios

Los usuarios pueden ser creados desde la parte de administración.

Cuando se crea un usuario, el backend se encarga de validar que:

- El email no esté repetido
- La contraseña cumpla las reglas de seguridad
- El rol sea válido
- Los datos lleguen con el formato correcto

Una vez creado:

- La contraseña se guarda en hash
- El usuario queda registrado en base de datos
- Se le asigna su rol correspondiente
- Puede quedar activo o pendiente de activación, según cómo se haya definido en la creación

Activación y desactivación

Uno de los puntos importantes del sistema es que un usuario puede estar creado pero no habilitado para usar la aplicación.

Esto se controla con el campo enabled.

¿Qué permite esto?

- Crear cuentas antes de que entren en uso
- Bloquear acceso sin borrar usuarios
- Desactivar temporalmente cuentas

Es una forma sencilla y muy útil de tener control sin perder información.

Cambio de rol

El sistema distingue entre usuarios normales y administradores.

Esto permite que desde administración se pueda cambiar el rol de una cuenta cuando sea necesario.

Ejemplo:

- Un usuario normal pasa a administrador
- Un admin deja de tener privilegios especiales

Este cambio debe hacerse con cuidado, porque afecta directamente a los permisos del sistema.

Edición de datos del usuario

El backend permite actualizar ciertos datos del usuario, como por ejemplo:

- Nombre
- Email
- Contraseña
- Avatar o foto de perfil

En cada caso se aplican validaciones para evitar errores o conflictos.

Por ejemplo:

- Si se cambia el email, se comprueba que no esté ya en uso
- Si se cambia la contraseña, debe cumplir la política de seguridad
- Si se cambia el nombre, se valida el formato básico

Qué puede hacer un usuario y qué puede hacer un admin

Aquí entra en juego la separación de permisos.

Usuario normal

Puede gestionar sus propios datos y usar las funciones normales de la aplicación, como fichar o consultar su resumen.

Administrador

Tiene acceso a funciones de gestión más amplias, como:

- Crear usuarios
- Activar o desactivar cuentas

- Cambiar roles
- Consultar otros usuarios
- Revisar fichajes abiertos
- Realizar ciertas acciones especiales sobre registros

Esto no se deja al frontend solamente. El backend también lo valida para que nadie pueda saltarse permisos manipulando peticiones.

ROLES Y PERMISOS

Resumen de las funcionalidades disponibles según el tipo de usuario.

 USER Usuario normal	 ADMIN Administrador
 Fichar entrada / salida  Ver su resumen de horas  Ver sus fichajes  Editar su perfil  Cambiar contraseña  Exportar sus datos (PDF / CSV)	 Todas las funciones de USER  Crear usuarios  Activar / desactivar usuarios  Cambiar roles  Ver todos los fichajes  Forzar cierre de fichajes  Crear fichajes manuales  Editar fichajes  Anular fichajes  Ver resumen global  Exportar datos globales (PDF / CSV)  Ver auditoría

Validaciones importantes

En la gestión de usuarios se aplican varias comprobaciones para mantener la integridad del sistema:

- Email único
- Contraseña segura
- Rol válido
- Usuario autenticado para editar su perfil
- Usuario administrador para acciones de gestión global

Esto evita problemas típicos como duplicados, accesos indebidos o datos inconsistentes.

Perfil de usuario

Además de la parte puramente administrativa, el sistema también contempla el perfil del usuario.

Desde ahí se pueden gestionar datos personales básicos, y en el futuro se puede ampliar con más información, como por ejemplo una foto de perfil más completa o elementos visuales para diferenciar mejor a los usuarios.

Esto es especialmente útil cuando hay varias personas con nombres parecidos dentro de la empresa.

7. Fichajes (control horario)

El sistema de fichajes es el núcleo de la aplicación.

Se encarga de registrar cuándo un usuario empieza y termina su jornada, y de gestionar toda la información asociada a ese proceso.

Pero no es solo “guardar horas”, sino que incluye control, validaciones, seguridad y trazabilidad.

Qué es un fichaje

Un fichaje representa un periodo de trabajo.

Está compuesto por:

- startAt → hora de inicio
- endAt → hora de fin (puede ser null)

- Si endAt es null → el fichaje sigue abierto.

Inicio de fichaje

Cuando un usuario inicia su jornada:

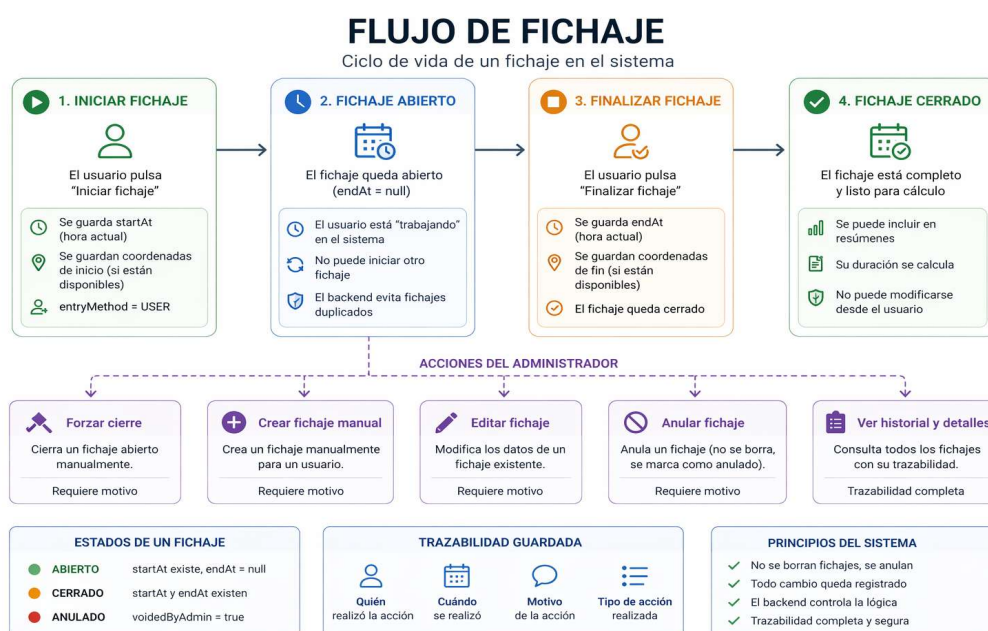
1. Pulsa el botón de fichar en el frontend
2. Se envía una petición a /api/fichaje/start
3. El backend:
 - Verifica el usuario (token)
 - Comprueba que no tenga ya un fichaje abierto
 - Guarda el nuevo registro con la hora actual

- A partir de ahí, el usuario está “trabajando” en el sistema.

Fin de fichaje

Cuando el usuario termina:

1. Pulsa “finalizar fichaje”
2. Se llama a /api/fichaje/stop
3. El backend:
 - Busca el fichaje abierto
 - Guarda la hora de fin (endAt)



Control de errores comunes

El sistema evita situaciones típicas como:

- Iniciar varios fichajes a la vez
- Cerrar fichajes que no existen
- Manipular tiempos desde el frontend

- Todo esto se controla desde el backend.

Consulta de estado

El endpoint `/api/fichaje/status` permite saber:

- Si el usuario está fichado o no
- Si tiene un fichaje abierto
- Información básica del estado actual

- Esto se usa para actualizar la interfaz en tiempo real.

Resumen de horas

El sistema permite obtener un resumen del tiempo trabajado:

- Por rango de fechas
- Con total de horas
- Días trabajados
- Detalle por fichaje

- Esto se usa tanto para el usuario como para el administrador.

Acciones de administrador

Los administradores tienen control adicional sobre los fichajes:

- Forzar cierre de fichajes abiertos
- Crear fichajes manuales
- Editar fichajes existentes
- Anular fichajes

Cada una de estas acciones:

- queda registrada
- requiere motivo (reason)
- afecta al historial del registro

Trazabilidad del fichaje

Cada fichaje guarda información sobre lo que ha ocurrido con él:

- Si fue creado por un admin
- Si fue editado
- Si fue anulado
- Quién hizo la acción
- Cuándo ocurrió

- Esto permite reconstruir todo el historial del registro.

Tipos de fichaje

El sistema distingue cómo se ha creado un fichaje:

- USER → creado por el usuario
- ADMIN → creado manualmente

Esto se guarda en el campo `entryMethod`.

Anulación de fichajes

En lugar de borrar registros, el sistema permite anularlos.

¿Por qué?

- No se pierde información
- Se mantiene historial
- Es más seguro

Se usan campos como:

- `voidedByAdmin`
- `voidReason`
- `voidedAt`

Edición de fichajes

Cuando un fichaje se modifica:

- Se guarda quién lo editó
- Se guarda el motivo
- Se registra la fecha de edición

Esto evita cambios “invisibles”.

Decisiones importantes

- No se borran fichajes → se anulan
- Todo cambio importante queda registrado
- El backend controla la lógica (no el frontend)
- Se evita duplicidad o estados inconsistentes

8. Geolocalización

El sistema de fichajes incluye la posibilidad de guardar la ubicación desde la que se realiza cada acción. Esto permite añadir contexto a los registros y tener una referencia aproximada de dónde se ha fichado.

Qué se guarda

En cada fichaje se pueden almacenar datos de geolocalización tanto al inicio como al final:

- `latStart` / `lonStart` → ubicación al iniciar fichaje
- `latEnd` / `lonEnd` → ubicación al finalizar fichaje
- `accuracyStart` / `accuracyEnd` → precisión de la ubicación

Estos datos son opcionales, ya que dependen de si el dispositivo del usuario permite obtener la ubicación.

Cómo funciona

El proceso es sencillo:

1. El frontend solicita la ubicación del usuario
2. Si el usuario acepta, se obtienen las coordenadas
3. Se envían junto con la petición de fichaje
4. El backend las guarda en la base de datos

- Si no hay ubicación disponible, el fichaje se sigue registrando igualmente.

Para qué sirve

La geolocalización no se usa para bloquear fichajes, sino para aportar información adicional.

Permite:

- Ver desde dónde se ha fichado
- Mostrar la ubicación en mapas
- Añadir contexto en revisiones o auditorías

- Es una capa informativa, no restrictiva.

Uso en la aplicación

Estos datos se utilizan principalmente en:

- Vista de fichajes
- Resúmenes
- Visualización en mapas (por ejemplo con Leaflet)

- Esto facilita entender mejor cada registro.

Consideraciones importantes

- La ubicación depende del dispositivo del usuario
- Puede no ser precisa al 100%
- Requiere permiso del usuario
- No siempre estará disponible

- Por eso se trata como un dato opcional.

Privacidad

Dado que se trabaja con datos de ubicación, se han tenido en cuenta aspectos básicos de privacidad:

- El usuario debe aceptar el uso de su ubicación
- Se guarda solo la información necesaria
- No se usa para seguimiento en tiempo real
- Se utiliza únicamente en el contexto del fichaje

Decisión de diseño

Se ha optado por incluir la geolocalización como un complemento, no como una obligación.

- Esto permite:

- No bloquear el uso de la aplicación
- Mantener flexibilidad
- Aportar valor sin complicar la experiencia

9. Auditoría del sistema

El sistema incluye un mecanismo de auditoría para registrar acciones importantes realizadas dentro de la aplicación, especialmente por parte de administradores.

La idea es simple:

- Saber qué ha pasado
- Quién lo ha hecho
- Y cuándo ha ocurrido

Qué es la auditoría

La auditoría consiste en guardar registros de acciones críticas en la base de datos.

Esto se hace mediante el modelo AdminAuditLog.

Cada vez que ocurre una acción importante, se genera un registro.

Qué se guarda en cada log

Cada entrada de auditoría incluye información como:

- adminUserId → quién realizó la acción
- action → qué acción se hizo
- targetType → sobre qué tipo de elemento (usuario, fichaje...)
- targetId → identificador del elemento afectado
- metadata → información adicional (opcional)
- ip → dirección IP (si está disponible)
- userAgent → información del navegador/dispositivo
- createdAt → fecha de la acción

- Esto permite tener un registro bastante completo.

Cuándo se registra una acción

No todas las acciones generan auditoría, solo las más relevantes.

Ejemplos típicos:

- Forzar cierre de un fichaje
- Editar un fichaje
- Anular un fichaje
- Crear fichajes manuales
- Acciones importantes sobre usuarios

- En general, cualquier acción que modifique datos de forma crítica.

Auditoría como parte del mantenimiento final

En la fase final del backend se revisó que las acciones administrativas importantes quedasen correctamente controladas.

La auditoría se mantiene como una capa de trazabilidad para acciones como:

- Crear fichajes manuales
- Editar fichajes
- Anular fichajes
- Forzar cierres
- Acciones relevantes de administración

La idea no es registrar absolutamente todo, sino guardar las acciones que realmente pueden afectar a la integridad del sistema.

Esto permite revisar qué ocurrió, quién lo hizo y en qué momento.

Para qué sirve

La auditoría es útil en varios casos:

- Revisar cambios realizados
- Detectar posibles errores
- Analizar comportamientos
- Tener trazabilidad en el sistema

- Es clave en entornos reales donde hay varios usuarios y administradores.

Ejemplo práctico

Imagina que un fichaje ha sido modificado.

Gracias a la auditoría se puede ver:

- Qué admin lo modificó
- Cuándo lo hizo
- Qué tipo de acción fue
- Información adicional si se guardó en metadata

- Esto evita situaciones de “no se sabe qué ha pasado”.



Relación con seguridad

La auditoría no solo es para control interno, también es una medida de seguridad.

Permite:

- Detectar acciones sospechosas
- Revisar accesos indebidos
- Tener evidencia en caso de problemas

Decisión de diseño

Se ha optado por registrar las acciones en una tabla separada en lugar de mezclarlo con otras entidades.

- Ventajas:

- Mantiene los datos organizados
- Facilita consultas de auditoría
- Permite escalar el sistema de logs

Relación con otros módulos

La auditoría está conectada principalmente con:

- Gestión de usuarios
- Gestión de fichajes
- Acciones administrativas

Actúa como una “capa invisible” que registra lo que ocurre.

10. Integridad de datos (hash)

El sistema incluye un mecanismo para detectar si un fichaje ha sido manipulado directamente en la base de datos.

La idea es sencilla:

- Si alguien cambia un fichaje por fuera del sistema
- el backend puede detectarlo

Qué es el hash en este contexto

Cada fichaje puede tener un campo hash.

Este hash se genera a partir de datos importantes del propio fichaje, por ejemplo:

- id
- userId
- startAt
- endAt
- otros datos relevantes
- y una clave secreta definida en el backend

- Todo eso se combina y se genera un valor único.

Cómo funciona

El sistema de integridad se basa en generar un hash del fichaje usando datos importantes del registro y una clave interna del backend.

El hash se actualiza cuando el fichaje cambia mediante el sistema.

Actualmente se aplica en acciones como:

- Creación manual de fichajes por administrador
- Edición de fichajes
- Anulación de fichajes
- Cierre de fichajes

Cada vez que el backend modifica un fichaje de forma válida, recalcula el hash y lo guarda.

Si más adelante los datos del fichaje no coinciden con su hash, el sistema puede detectar que ese registro ha sido manipulado directamente en la base de datos.

Detección de manipulación

Cuando se detecta una diferencia:

- Se puede marcar el fichaje como comprometido
- Se guarda la fecha en `tamperDetectedAt`

- Esto permite saber que ese registro no es fiable.

Para qué sirve

Este sistema añade una capa extra de seguridad:

- Detecta cambios hechos directamente en la base de datos
- Evita manipulaciones externas
- Aumenta la confianza en los datos

- Es especialmente útil en sistemas donde los registros tienen valor legal o laboral.

Ejemplo práctico

Imagina que alguien accede a la base de datos y cambia:

- La hora de salida (`endAt`)

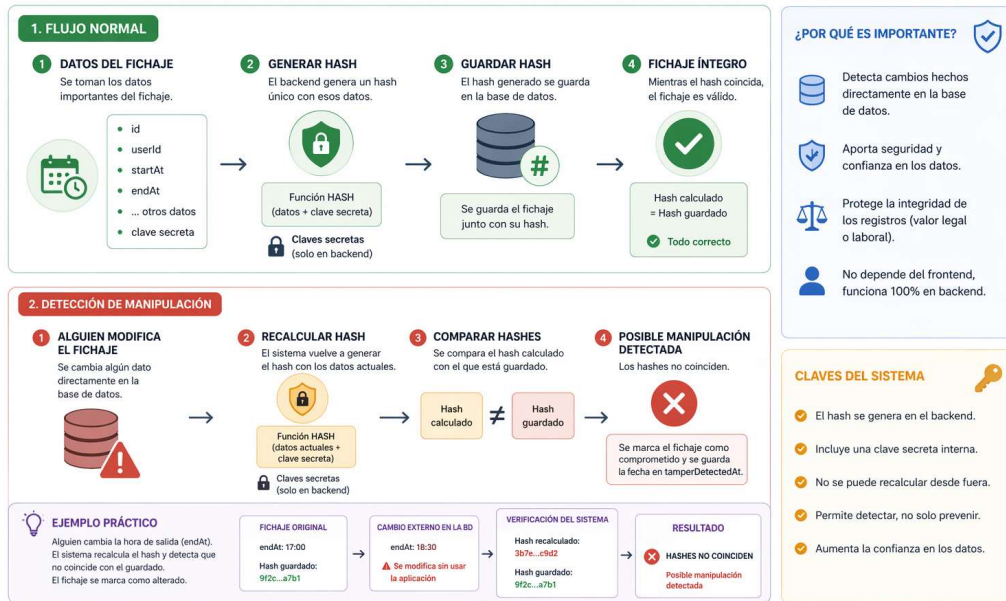
El sistema:

1. Recalcula el hash
2. Detecta que no coincide
3. Marca el fichaje como alterado

- Así no pasa desapercibido.

INTEGRIDAD DE DATOS (HASH)

Cada fichaje tiene un hash que permite detectar manipulaciones en la base de datos.



Por qué es importante la clave secreta

El hash incluye una clave interna del backend.

Esto es clave porque:

- Evita que alguien pueda recalcular el hash desde fuera
- Hace que el sistema sea realmente seguro

Decisión de diseño

Se ha optado por este sistema porque:

- Es ligero (no afecta mucho al rendimiento)
- Añade seguridad sin complicar la app
- No depende del frontend
- Funciona completamente en backend

Relación con otros módulos

Este sistema está directamente relacionado con:

- Fichajes (TimeEntry)
- Auditoría
- Seguridad del sistema

- Refuerza la integridad de los datos en todo el flujo.

11. Exportaciones (PDF / CSV)

La aplicación permite exportar datos en formatos estándar como **PDF** y **CSV**, facilitando su uso fuera del sistema.

Esto es especialmente útil para:

- Informes
- Control interno
- Envío a terceros (gestoría, empresa, etc.)

Tipos de exportación

Actualmente, el sistema permite exportar principalmente información relacionada con los fichajes. Dependiendo del rol del usuario:

Usuario normal

Puede exportar:

- Sus propios datos
- Su resumen de horas

Administrador

Puede exportar:

- Datos globales
- Fichajes de todos los usuarios
- Resúmenes completos

- Esto permite tener una visión general del sistema.

Cómo funciona

El proceso de exportación es sencillo:

1. El usuario selecciona los datos (por ejemplo, un rango de fechas)
2. El frontend envía la petición al backend
3. El backend:
 - Obtiene los datos de la base de datos
 - Los procesa
 - Devuelve el archivo generado

- El usuario puede descargarlo directamente.

Exportación en CSV

El formato CSV (valores separados por comas) permite:

- Abrir los datos en Excel o herramientas similares
- Trabajar con hojas de cálculo
- Analizar información fácilmente

- Es el formato más práctico para tratamiento de datos.

Exportación en PDF

El PDF está pensado como documento visual:

- Resúmenes claros

- Información organizada
- Listo para imprimir o compartir

- Ideal para informes o documentación.

Dónde se genera el archivo

En este caso, la generación del archivo se realiza principalmente en el frontend, utilizando librerías específicas.

- El backend se encarga de:

- Proporcionar los datos
- Validar permisos
- Asegurar que la información sea correcta

Control de acceso

El backend controla qué puede exportar cada usuario:

- Un usuario solo puede exportar sus propios datos
- Un administrador puede exportar información global

- Esto evita accesos indebidos a datos sensibles.

Consideraciones

- El volumen de datos puede afectar al rendimiento
- Es importante validar bien los filtros (fechas, usuarios...)
- Los datos exportados deben ser coherentes con los mostrados en la aplicación

12. Flujo general de la aplicación

Hasta ahora hemos visto cada parte por separado: usuarios, fichajes, seguridad, auditoría...

En este apartado se explica cómo encaja todo en el día a día de la aplicación.

La idea es ver el sistema como un todo.

Flujo básico de uso

El uso normal de la aplicación sigue este recorrido:

1. El usuario inicia sesión
2. El sistema valida sus credenciales
3. Se genera un token de acceso
4. El usuario accede a la aplicación
5. Realiza acciones (fichar, consultar, etc.)
6. El backend procesa cada petición
7. Se guardan los datos en la base de datos
8. Se devuelven los resultados al frontend

- Todo pasa siempre por el backend.

Qué ocurre en cada petición

Cada vez que el frontend hace una llamada:

1. Se envía el token JWT
2. El backend valida al usuario
3. Comprueba permisos (USER / ADMIN)
4. Ejecuta la lógica correspondiente
5. Accede a la base de datos con Prisma
6. Devuelve la respuesta

- Este flujo es común a toda la aplicación.

Ejemplo completo (fichaje)

Vamos a ver un caso real completo:

Inicio de jornada

- El usuario pulsa "Fichar"
- Se envía petición al backend
- Se valida el token
- Se crea un fichaje (startAt)
- Se guarda en base de datos

Fin de jornada

- El usuario pulsa "Finalizar fichaje"
- Se busca el fichaje abierto
- Se guarda endAt
- Se cierra el registro

Consulta

- El usuario pide su resumen
- El backend calcula los datos
- Se devuelven al frontend

Flujo con administrador

Cuando entra un administrador, el flujo es el mismo, pero con más capacidades:

- Puede gestionar usuarios
- Puede ver todos los fichajes
- Puede modificar registros
- Puede generar auditoría

- Pero siempre pasando por validaciones de backend.

Seguridad integrada en el flujo

Durante todo el proceso se aplican medidas de seguridad:

- Validación de token en cada petición
- Control de permisos
- Validación de datos
- Hash de integridad en fichajes
- Registro en auditoría

- La seguridad no es un paso aparte, está integrada en todo el flujo.

Papel del frontend y backend

Frontend

- Muestra la información
- Recoge acciones del usuario
- Envía peticiones

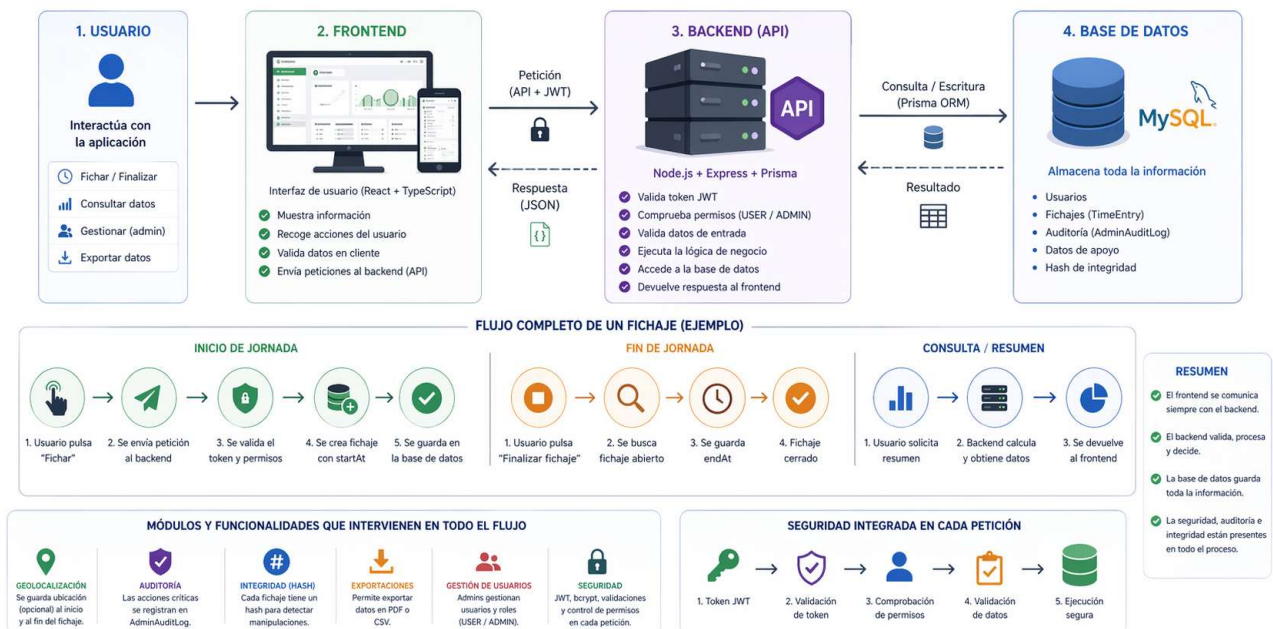
Backend

- Valida
- Decide
- Ejecuta
- Guarda

- El backend es el que tiene el control real.

FLUJO GENERAL DE LA APLICACIÓN

Visión completa de cómo interactúan los usuarios, el frontend, el backend y todos los módulos del sistema.



13 .Decisiones técnicas

Durante el desarrollo del backend se han tomado varias decisiones técnicas con el objetivo de crear una aplicación que no solo funcione, sino que sea mantenible, segura y fácil de ampliar en el futuro. En este apartado se explican las más importantes.

Uso de API con Next.js

Se ha utilizado Next.js como backend aprovechando su sistema de rutas.

¿Por qué?

- Permite tener backend y frontend desacoplados
- Facilita crear endpoints rápidamente
- Es una solución moderna y muy utilizada

Uso de TypeScript

El backend está desarrollado con TypeScript.

¿Por qué?

- Reduce errores
- Mejora la legibilidad del código
- Facilita el mantenimiento

Uso de Prisma como ORM

Se ha elegido Prisma para gestionar la base de datos.

¿Por qué?

- Permite trabajar con objetos en lugar de SQL
- Hace el código más limpio
- Facilita cambios en el modelo de datos

Uso de PostgreSQL

La base de datos utilizada es PostgreSQL.

¿Por qué?

- Es robusta y fiable
- Muy utilizada en entornos profesionales
- Soporta bien relaciones complejas

Autenticación con JWT

Se ha implementado autenticación mediante tokens JWT usando jsonwebtoken.

¿Por qué?

- Permite sesiones sin estado
- Es fácil de integrar
- Funciona bien con APIs

Hash de contraseñas con bcrypt

Las contraseñas se almacenan usando bcrypt.

¿Por qué?

- Evita guardar contraseñas en texto plano
- Aumenta la seguridad del sistema

Sistema de auditoría

Se ha implementado un sistema de logs mediante el modelo AdminAuditLog.

¿Por qué?

- Permite trazabilidad
- Facilita el control de acciones
- Mejora la seguridad

Integridad de datos con hash

Se ha añadido un sistema de hash en los fichajes.

¿Por qué?

- Detecta manipulaciones externas
- Aumenta la confianza en los datos
- Añade una capa extra de seguridad

Control de fichajes sin borrado

Los fichajes no se eliminan, se anulan.

¿Por qué?

- No se pierde información
- Se mantiene historial
- Es más seguro en entornos reales

Geolocalización opcional

La geolocalización se ha implementado como dato opcional.

¿Por qué?

- No bloquea al usuario
- Aporta contexto sin complicar el sistema
- Respeto la privacidad

Exportaciones en frontend

La generación de PDF y CSV se realiza en el frontend.

¿Por qué?

- Mayor flexibilidad visual
- Menor carga en el backend
- Mejor experiencia de usuario

Separación frontend / backend

Se ha separado claramente el frontend (Angular) del backend.

¿Por qué?

- Permite escalar cada parte por separado
- Facilita mantenimiento
- Es una arquitectura estándar

Uso de selectores reutilizables

Se decidió centralizar la forma en la que se devuelven los datos de usuario mediante selectores reutilizables. Esto evita que cada endpoint decida por separado qué campos devuelve.

Ventajas:

- Menos errores entre backend y frontend
- Respuestas más consistentes
- Mayor control sobre qué datos se exponen
- Mejor mantenimiento a futuro

Especialmente se usan:

`userPublicSelect`

`userAdminSelect`

Esta decisión ayuda a separar datos públicos de datos administrativos y prepara el sistema para crecer sin desordenar las respuestas.

Reorganización interna del backend

Otra decisión importante fue reorganizar la carpeta `src/lib` por responsabilidades.

En lugar de tener utilidades mezcladas, se separaron en bloques como:

- Autenticación
- Auditoría
- Correos
- Seguridad
- Selectores
- Prisma
- Utilidades HTTP

Esto hace que el backend sea más limpio, más escalable y más fácil de entender para cualquier compañero que entre al proyecto.

Enfoque general del proyecto

Se ha buscado un equilibrio entre:

- Simplicidad
- Funcionalidad
- Seguridad
- Escalabilidad

Sin complicar en exceso la estructura, pero aplicando buenas prácticas reales.

14. Posibles mejoras

Aunque la aplicación ya cumple su objetivo principal, existen varias mejoras que podrían implementarse en el futuro para hacer el sistema más completo, escalable y preparado para un entorno real de empresa.

Mejora del perfil de usuario

Se podría ampliar la gestión del perfil añadiendo:

- Subida de imagen/avatar directamente desde la aplicación
- Datos adicionales del usuario (departamento, puesto, etc.)
- Mejor personalización visual

- Esto ayudaría a diferenciar mejor a los usuarios y mejorar la experiencia.

Búsqueda y filtros avanzados

En la parte de administración, sería útil añadir:

- Búsqueda por nombre o email
- Filtros por rol o estado (activo/inactivo)
- Paginación de resultados

- Esto es clave cuando hay muchos usuarios en el sistema.

Mejoras en informes y exportaciones

Se podrían mejorar las exportaciones actuales:

- Añadir más detalle en los PDFs
- Incluir información geográfica en texto (dirección aproximada)
- Crear informes más completos para administradores

- Esto aumentaría el valor del sistema para uso real.

Mejora de la geolocalización

La geolocalización podría evolucionar hacia:

- Mostrar direcciones en lugar de coordenadas
- Mejorar la visualización en mapas
- Añadir validaciones por zona (opcional)

- Siempre manteniendo el equilibrio con la privacidad.

Mejora de la seguridad

Posibles mejoras en seguridad:

- Expiración y renovación de tokens (refresh tokens)
- Uso de HTTPS en entorno de producción
- Protección adicional en endpoints sensibles
- Registro más detallado de accesos

- Esto haría el sistema más robusto en producción.

Mejora del sistema de auditoría

El sistema de auditoría podría ampliarse con:

- Más tipos de acciones registradas
- Visualización avanzada en frontend
- Filtros y búsqueda en logs

- Facilitaría aún más el control del sistema.

Optimización del backend

A nivel técnico, se podrían añadir:

- Paginación en endpoints grandes
- Filtros avanzados en consultas
- Mejora de rendimiento en consultas complejas

- Esto es importante si la aplicación crece.

Adaptación a otros entornos

La aplicación podría ampliarse a:

- Versión móvil nativa
- Mejor adaptación a tablets
- Posible uso como app empresarial

- Esto abriría nuevas posibilidades de uso.

Automatización y notificaciones

Se podrían añadir:

- Avisos de fichajes olvidados
- Notificaciones automáticas
- Recordatorios

- Mejora la experiencia del usuario.

Escalabilidad futura

Pensando a largo plazo:

- Separar backend en servicios (si crece mucho)
- Mejorar arquitectura para alta carga
- Integración con otros sistemas (ERP, RRHH...)

Esto prepararía el sistema para un uso empresarial real.

Enfoque de mejora

Todas estas mejoras siguen una idea clara:

- Mejorar sin romper lo que ya funciona
- Mantener el sistema simple pero potente
- Evolucionar según necesidades reales